

Lab Assignment 4: How to Use APIs in Python

DS 6001

Instructions

Please answer the following questions as completely as possible using text, code, and the results of code as needed. Format your answers in a Jupyter notebook. To receive full credit, make sure you address every part of the problem, and make sure your document is formatted in a clean and professional way.

Problem 0

Import the following packages:

```
In [20]: import numpy as np
import pandas as pd
import requests
import json
import os
import dotenv
import lyricsgenius
```

We will use the following user-agent:

```
In [21]: mybot = 'ds6001'
botversion = '0.0'
botwebsite = 'Data access for a class assignment' # description because we h
email = 'kng5vf@virginia.edu' # change this to your own email, or leave it a
software = 'python-requests'
softwareversion = requests.__version__

useragent = f'{mybot}/{botversion} ({botwebsite}; {email}) {software}/{softw
useragent
```

```
Out[21]: 'ds6001/0.0 (Data access for a class assignment; kng5vf@virginia.edu) pytho
n-requests/2.34.2'
```

Problem 1

The hardest part of working with an API is finding and understanding the API's documentation. While the documentation (if it exists!) is essential for helping you to navigate to the data you need, they are also dense and technical.

One big reason why API documentation is difficult is that it has to speak to all use-cases. Very often, when software engineers interact with an API like genius.com's, they are building apps. Imagine that you are hired to build an events and ticketing app for a local music venue. You might want to embed a list of albums, songs, and background information in your app by connecting to the Genius API (for song lyrics and artist information) and pulling that data into your app. You might run a record label and you want to interact with this API by posting information about the musicians you've signed. Or you might want to collect data on the users of genius.com for marketing research. The documentation must provide enough information to support all of these uses. As a result, documentation tends to be very long, and most of the information you will see is neither relevant nor easy to understand. That is okay, and should be the expectation.

To effectively use API documentation, take a deep breath. Then narrow your search through the documentation to only these four things:

1. **The root:** We make requests from an API by, in part, writing out a complicated URL. In Python, we will compile the segments of this URL and let the `requests` package build the right URL. The first segment is the root, or the shared beginning of every URL associated with this API. Roots sometimes also include a version number such as `/v2` or `/v3`. Your first task is to find the root and save it as a string in Python. One complaint I have about API documentation is that sometimes the root is a bit difficult to find, and that's crazy because I think it should be written in bold letters on the homepage. Sometimes I have to look for *example calls* to the API to see the root being used.
2. **The endpoints:** The next part of an API call's URL (after the root but before the question mark) is called the endpoint. Very commonly, APIs have many endpoints for many different types of data. Genius for example has separate endpoints for songs, artists, user accounts, its search engine, and other types of functions. Sometimes endpoints are called "resources" or something similar, but you can generally tell an endpoint because it is denoted with a slash first, such as Genius's `/songs`, `/artists`, `/accounts`, and `/search` sections under resources. Your task here is to find the endpoint or endpoints that can give you the data you need by reading their descriptions in the documentation.
3. **The endpoint parameters:** Some endpoints will also have a list of parameters that control the selection of data a request will return. There are two ways that these parameters can appear in an API call's URL. First, they can appear as "query parameters" after a question mark, using equal signs to specify parameter values, and separating parameters with & signs. If you write these parameters in a dictionary and use the `params` argument in the `requests.get()` method to supply these parameters to the API, then `requests` will format the URL properly for you. The other way parameters might appear is as "path parameters" which are part of the endpoint, sometimes surrounded by curly braces: this format implies

that the parameter value should be part of the endpoint you supply to `requests.get()` instead of an entry in the `params` argument. Either way, your task here is to find the parameters that enable you to get the data you need and to use the correct names and values of these parameters in your Python code.

4. How to acquire API authentication or access keys, and how to supply these credentials when interacting with the API: It's important to remember that APIs are designed to give the data owners control over how their data is shared with users like you. There is a political dimension to that: a data owner can choose what to divulge or keep private. There's also a financial dimension: websites can charge you at varying rates for using the API, can restrict the speed and volume of your requests to prevent you from overloading systems or profiting too much off of their work, or can use a "freemium" model where only a small amount of data is free but more data carries charges. Many APIs require you to register in advance and acquire secret codes (called keys or tokens) that must be supplied back to the API to enable your use. You have two tasks here. First you need to find instructions in the documentation for acquiring API access keys, paying particular attention to whether they are asking for a credit card (Genius does not) and if so what they might charge you, and what limits exist on the amount of data you can grab. Second, you need to find the instructions for supplying your key to the API. Sometimes that will be one of the endpoint parameters, and sometimes it will be in the headers along with your user-agent string. (When providing an API key via the headers, there's a common format that requires you to write the word "Bearer" and a space before your key. You are likely to see this format pop up sometimes.)

For the following problems, describe in words how you can get the data in question. Use the API documentation linked in each question. Be specific about

- the root,
- endpoint,
- parameters,
- method of acquiring keys,
- method for supplying these keys during an API call,
- and whether and how using the API costs money, or is limited to only certain quantities of data.

If there is more than one way to accomplish a task, describing just one of the valid approaches is fine.

If you use an LLM/gen AI to read the API documentation, prompt it to identify the root, endpoint, relevant parameters, and the mechanism for acquiring and supplying API keys. Ask for the specific parts of the documentation that supply these answers so you can double check. If instead you prompt something vague and general ("how do I access this

API?") you will have a high likelihood of getting slop that either fails to work, or in the worst cases, exposes your keys or charges your credit card in ways you don't foresee.

Part a

Do not write Python code to accomplish these steps. Focus on reading the documentation and understanding what you would need to do if you did attempt to get the data. Also, do NOT give any of these websites a credit card or other payment information, and please do not copy and paste your API keys into this document.

How would you get data on the top 50 headlines from Germany today on the topic of sports? Use the News API, with documentation here: <https://newsapi.org/docs> (Hint: here's a list of country codes: https://en.wikipedia.org/wiki/ISO_3166-1_alpha-2) [12 points]

Answer

If we want to request the top 50 sports headlines from Germany using the News API, we would use the root <https://newsapi.org/v2>, which can be found in the examples provided in the documentation. The endpoint we want is /top-headlines, which is listed under the endpoints section. On the same page, we can find the parameters we need. We would set country=de for Germany, category=sports to get sports news, and pageSize=50 to request 50 headlines. To get an API key, we would need to sign up for a free account on the News API website. Once we have our key, we can provide it in two ways, as the apiKey query parameter or in the request header using X-API-Key. The free plan allows up to 100 requests per day with a 24-hour delay.

Part b

Again, do not write Python code to accomplish these steps. Focus on reading the documentation and understanding what you would need to do if you did attempt to get the data. Also, do NOT give any of these websites a credit card or other payment information, and please do not copy and paste your API keys into this document.

How would you get JSON formatted data on the 100 most recent house resolutions (bills that passed a vote in the House of Representatives) in the 119th congress (elected in 2024) from the API of the U.S Congress? The documentation for this API is here: <https://api.congress.gov/> [12 points]

Answer

To get the 100 most recent house resolutions in the 119th Congress, the Congress.gov API provides us with the root <https://api.congress.gov/v3>, which I assume is the root after looking at the examples they provide. They provide many endpoints, but the one we want is /bill/{congress}/{billType}, where we can provide 119 for Congress and hres for

House resolutions. For the parameters, we could set `format=json` to get the data in JSON format. Also, setting `limit=100` would allow us to get 100 results. To get an API key, we would need to sign up for a free account. Once we have our key, we can supply it as a query parameter named `api_key`. The API has a limit of 5,000 requests per hour, and each request can return a maximum of 250 results. Since we only need 100, we could retrieve them in one request.

Part c

Once more, do not write Python code to accomplish these steps. Focus on reading the documentation and understanding what you would need to do if you did attempt to get the data. Also, do NOT give any of these websites a credit card or other payment information, and please do not copy and paste your API keys into this document.

How would you get a real-time property value estimate for a specific address, say 1 [Lewis Mountain Parkway, Charlottesville, VA 22904](#) for example, using the RentCast API? The API documentation is here: <https://developers.rentcast.io/reference/introduction> When calculating costs, suppose I want to repeat this for each of 25,000 Charlottesville area addresses. [12 points]

Answer

To get a property value using the RentCast API, we would use the root <https://api.rentcast.io/v1>, which I found again by looking at an example, and the endpoint `/avm/value`, which can be found under the Property Valuation section of the documentation. For the parameters, we would provide the full address using `address=1 Lewis Mountain Parkway, Charlottesville, VA 22904` as describe it. We can also find a list of parameters on the same Property Valuation page. To get an API key, we would need to create a RentCast account. They offer a free plan, but it only includes 50 requests per month. Once we have our key, we would supply it in the request header using `X-API-Key` in the headers.

Problem 2

Next we'll work through the steps to access the Genius API and bring data from this API into Python.

Part a

Genius.com's API is documented here: <https://docs.genius.com/#/getting-started-h1>.

Follow the same process you used in problem 1 to find the information you need to use this API. Start with learning about how to use the search endpoint to do a search for "Bob Dylan". Write briefly your findings regarding the root, endpoint, parameters,

method of acquiring keys, and the method for supplying these keys during an API call. In this case, you are specifically looking for a credential called an Access Token. This API is free and has minimal restrictions, so no need to look for that information this time.

(Hint: because you are just trying to pull data, and not set up access for additional registered users, look for "Access for Apps Without Users" in the Authentication section of the API documentation. You can find the "API Client management page" by clicking "Manage Clients" on the left.) [12 points]

Answer

To briefly describe what we doing we are going to search for "Bob Dylan" using the Genius API, we would use the root <https://api.genius.com> and the endpoint /search. For the parameters, we would use q=Bob Dylan and to get an access token, we would need to create a Genius account. Once we have our access token, we would supply it in the request header using authorization.

Part b

Create a `.env` file for this project. The easiest way to do this in VS Code is to click on File -> New File, then name the file ".env" exactly, with no additional characters.

Open the .env file, copy your Genius access token into it with a name, and save it. Then use Python code to load this environmental variable and save it as a Python variable. (You can print the access token to make sure it worked, but do not allow your access token to display in your final notebook). [12 points]

```
In [22]: from dotenv import load_dotenv

load_dotenv()
genius_access_token = os.getenv("GENIUS_ACCESS_TOKEN")
```

```
In [23]: print(genius_access_token is not None)
```

True

Part c

Using the `requests` package in Python, connect to the Genius API's /search endpoint and search for "Bob Dylan". Remember to include a dictionary under `headers` that includes your user-agent string.

The output will be in JSON format. Genius's API is organized in a way that every individual artist has his or her own API endpoint with more detailed information about the artist. Display the portion of the JSON output that displays the API endpoint path for the data on Bob Dylan. (Hint: it is listed under `primary_artist` several branches down the JSON tree.) [14 points]

```
In [24]: mybot = 'ds6001_lab4_student_006'
botversion = '0.0'
botwebsite = 'learning the Genius API'
email = 'qrk9cs@virginia.edu'
software = 'python-requests'
softwareversion = requests.__version__

useragent = f'{mybot}/{botversion} ({botwebsite}; {email}) {software}/{softwareversion}'

root = "https://api.genius.com"
endpoint = "/search"
params = {"q": "Bob Dylan"}

myheaders = {
    "User-Agent": useragent,
    "Authorization": "Bearer " + genius_access_token
}

response = requests.get(root + endpoint, params=params, headers=myheaders)
print(response)

genius_json = response.json()
for hit in genius_json["response"]["hits"]:
    artist = hit["result"]["primary_artist"]

    if artist["name"] == "Bob Dylan":
        print(artist["name"])
        print(artist["api_path"])
        break
```

```
<Response [200]>
Bob Dylan
/artists/181
```

Part d

Add `/songs` to the end of the the endpoint path you found in part (c) and use this path to request the 20 most popular songs written by Bob Dylan (some of these may be performed by other artists because Dylan is one of the most frequently covered musicians of all time). Organize these data in a `pandas` data frame. [14 points]

```
In [25]: endpoint = "/artists/181/songs"
params = {
    "sort": "popularity",
    "per_page": 20
}

response = requests.get(root + endpoint, params=params, headers=myheaders)
print(response)

songs_json = response.json()
bob_dylan_df = pd.DataFrame(songs_json["response"]["songs"])
bob_dylan_df
```

<Response [200]>

Out [25] :

	annotation_count	api_path	artist_names	full_title	
0	18	/songs/96286	U.S.A. For Africa	We Are the World by U.S.A. For Africa	https://images.ger
1	3	/songs/68146	Adele	Make You Feel My Love by Adele	https://images.g
2	12	/songs/79424	Bob Dylan	Blowin' in the Wind by Bob Dylan	https://images.g
3	20	/songs/62605	Bob Dylan	The Times They Are A-Changin' by Bob Dylan	https://images.g
4	29	/songs/54784	Bob Dylan	Like a Rolling Stone by Bob Dylan	https://images.g
5	15	/songs/98664	Bob Dylan	All Along the Watchtower by Bob Dylan	https://images.g
6	7	/songs/103982	Guns N' Roses	Knockin' on Heaven's Door by Guns N' Roses	https://images.g
7	128	/songs/5393247	Bob Dylan	Murder Most Foul by Bob Dylan	https://images.
8	10	/songs/84846	Bob Dylan	Don't Think Twice, It's All Right by Bob Dylan	https://images.g
9	32	/songs/87402	Bob Dylan	Hurricane by Bob Dylan	https://images.g
10	22	/songs/224	Bob Dylan	Subterranean Homesick Blues by Bob Dylan	https://images.ge
11	5	/songs/199531	Bob Dylan	Knockin' on Heaven's Door by Bob Dylan	https://images.g
12	10	/songs/63341	The Jimi Hendrix	All Along the Watchtower	https://images.

	annotation_count	api_path	artist_names	full_title	
			Experience	by The Jimi Hendrix E...	
13	41	/songs/105186	Bob Dylan	A Hard Rain's A-Gonna Fall by Bob Dylan	https://images.g
14	37	/songs/118799	Bob Dylan	Tangled Up in Blue by Bob Dylan	https://images.g
15	10	/songs/79749	Bob Dylan	Make You Feel My Love by Bob Dylan	https://images.g
16	14	/songs/154239	Darius Rucker	Wagon Wheel by Darius Rucker	https://images.g
17	23	/songs/75446	Bob Dylan	Mr. Tambourine Man by Bob Dylan	https://images
18	82	/songs/92350	Bob Dylan	Desolation Row by Bob Dylan	https://images.g
19	51	/songs/75463	Bob Dylan	It's Alright, Ma (I'm Only Bleeding) by Bob Dylan	https://images.g

20 rows × 26 columns

Problem 3

One drawback of the Genius API is that it currently has no endpoint that returns song lyrics.

The `lyricsgenius` package is a wrapper around `requests` and `BeautifulSoup`, the Python package for web-scraping data off of the raw HTML of a webpage. We will cover web-scraping in module 5. But for now, follow the guide on the GitHub repository for this package (<https://github.com/johnwmillr/LyricsGenius>) for instructions on using it. Then use `lyricsgenius` to download and display the lyrics to "Tangled Up in Blue" by Bob Dylan.

Optional but recommended: call a loved one on the phone and sing "Tangled Up in Blue" to them. [12 points]

```
In [26]: import lyricsgenius

genius = lyricsgenius.Genius(genius_access_token)
song = genius.search_song("Tangled Up in Blue", "Bob Dylan")

if song is not None:
    print(song.lyrics)
else:
    print("Song not found.")
```

[Verse 1]

Early one morning the sun was shining
I was laying in bed
Wondering if she'd changed at all
If her hair was still red
Her folks they said our lives together
Sure was going to be rough
They never did like Mama's homemade dress
Papa's bankbook wasn't big enough
And I was standing on the side of the road
Rain falling on my shoes
Heading out for the East Coast
Lord knows I've paid some dues
Getting through
Tangled up in blue

[Verse 2]

She was married when we first met
Soon to be divorced
I helped her out of a jam, I guess
But I used a little too much force
We drove that car as far as we could
Abandoned it out west
Split up on a dark sad night
Both agreeing it was best
She turned around to look at me
As I was walking away
I heard her say over my shoulder
"We'll meet again someday
On the avenue"
Tangled up in blue

[Verse 3]

I had a job in the great north woods
Working as a cook for a spell
But I never did like it all that much
And one day the ax just fell
So I drifted down to New Orleans
Where I lucky was to be employed
Working for a while on a fishing boat
Right outside of Delacroix
But all the while I was alone
The past was close behind
I seen a lot of women
But she never escaped my mind
And I just grew
Tangled up in blue

[Verse 4]

She was working in a topless place
And I stopped in for a beer
I just kept looking at the side of her face
In the spotlight, so clear
And later on, when the crowd thinned out
I was just about to do the same
She was standing there, in back of my chair

Said, "Tell me, don't I know your name?"
I muttered something underneath my breath
She studied the lines on my face
I must admit, I felt a little uneasy
When she bent down to tie the laces
Of my shoe
Tangled up in blue

[Verse 5]
She lit a burner on the stove
And offered me a pipe
"I thought you'd never say hello," she said
"You look like the silent type"
Then she opened up a book of poems
And handed it to me
Written by an Italian poet
From the thirteenth century
And every one of them words rang true
And glowed like burning coal
Pouring off of every page
Like it was written in my soul
From me to you
Tangled up in blue

[Verse 6]
I lived with them on Montague Street
In a basement down the stairs
There was music in the cafes at night
And revolution in the air
Then he started into dealing with slaves
And something inside of him died
She had to sell everything she owned
And froze up inside
And when it finally, the bottom fell out
I became withdrawn
The only thing I knew how to do
Was to keep on keeping on
Like a bird that flew
Tangled up in blue

[Verse 7]
So now I'm going back again
I got to get to her somehow
All the people we used to know
They're an illusion to me now
Some are mathematicians
Some are carpenters' wives
Don't know how it all got started
I don't know what they're doing with their lives
But me, I'm still on the road
A-heading for another joint
We always did feel the same
We just saw it from a different point
Of view
Tangled up in blue